

Group B

Library Management System - Technical Documentation

Overview

This technical documentation provides an in-depth look at the architecture, components, and technologies used in the Library Management System. It is intended for developers and technical stakeholders who need to understand the system's design, structure, and underlying principles.

1. System Architecture

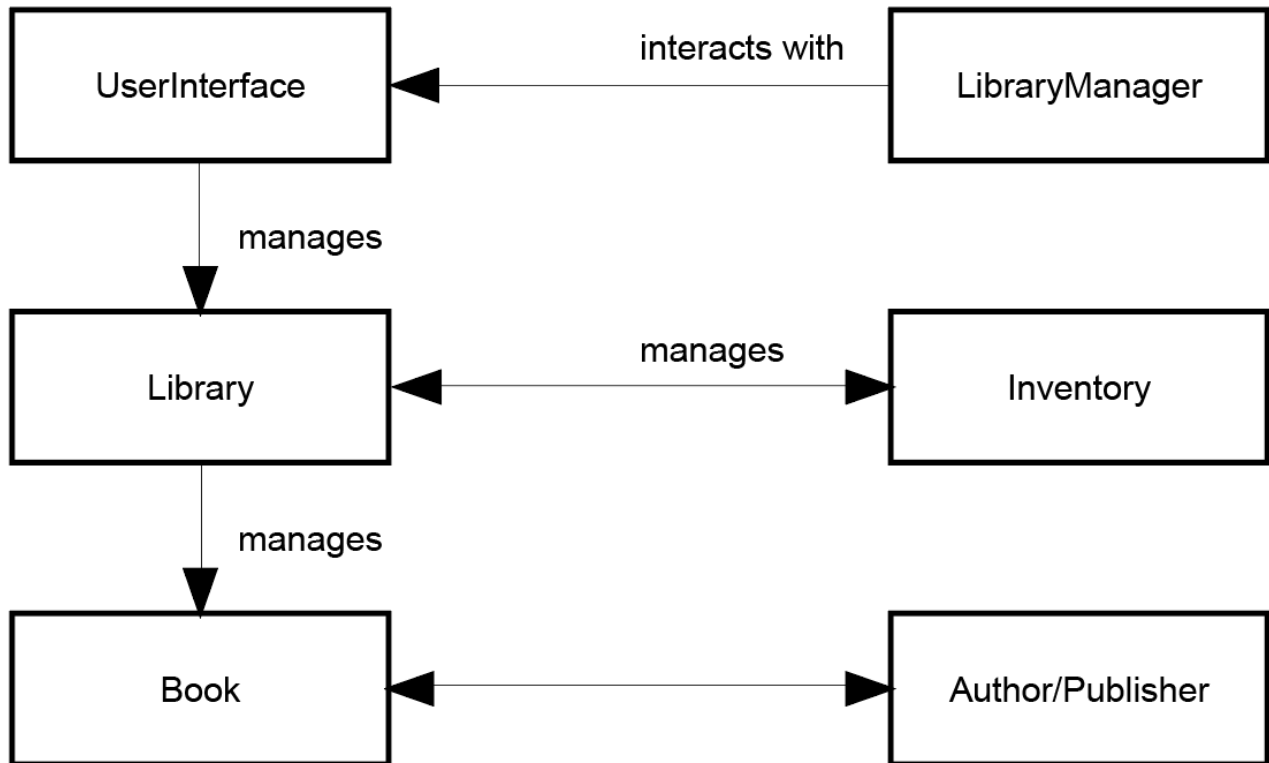
1.1 Layered Architecture

The Library Management System is built using a layered architecture, which separates different concerns into distinct layers. This design promotes modularity, maintainability, and scalability.

- **Presentation Layer:** This layer handles user interaction and input. It includes the ``UserInterface`` class, which is responsible for displaying menus, collecting user input, and showing messages to the user.
- **Business Logic Layer:** This layer contains the core functionality of the system. It includes the main operations for managing books, patrons, borrowings, returns, reservations, and more. Key classes in this layer are ``Library``, ``Book``, ``Patron``, ``Reservation``, ``Fine``, and ``LibraryEvent``.
- **Data Access Layer:** This layer manages data storage and retrieval. In this simplified system, the data is stored in memory using collections such as lists and maps. The ``Inventory`` class manages the quantities of books available in the library.

1.2 Component Diagram

The system consists of several key components that interact with each other:



2. Key Components

2.1 Library

The `Library` class is the central component of the system, responsible for managing all operations related to books, patrons, and events. It maintains collections of books, patrons, and reservations.

- **Attributes:-**

- `List<Book> books`: A list of all books in the library.
- `Map<String, Patron> patrons`: A map of all patrons, indexed by their unique ID.
- `Inventory inventory`: The library's inventory of books.
- `List<LibraryEvent> events`: A list of events hosted by the library.
- `PriorityQueue<Reservation> reservations`: A priority queue of reservations, ordered by the reservation date.

- **Key Methods:**

- ``addBook(Book book)``: Adds a book to the library's collection and updates the inventory.
- ``removeBook(Book book)``: Removes a book from the library's collection and updates the inventory.
- ``addPatron(Patron patron)``: Adds a new patron to the library's collection.
- ``removePatron(String patronId)``: Removes a patron from the library's collection.
- ``borrowBook(String patronId, String isbn)``: Allows a patron to borrow a book.
- ``returnBook(String patronId, String isbn)``: Allows a patron to return a borrowed book.
- ``searchBook(String isbn)``: Searches for a book by its ISBN.
- ``searchBooksByTitle(String title)``: Searches for books by their title.

2.2 Book

The ``Book`` class represents a book in the library. It includes information about the book's ISBN, title, author, publisher, and availability status.

- **Attributes:**

- ``String isbn``: The unique identifier for the book.
- ``String title``: The title of the book.
- ``Author author``: The author of the book.
- ``Publisher publisher``: The publisher of the book.
- ``boolean isAvailable``: Indicates whether the book is available for borrowing.

Key Methods:

- ``getDetails()``: Returns a string containing the book's details.
- ``compareTo(Book other)``: Compares this book with another based on their titles.

2.3 Patron

The `Patron` class represents a library patron who can borrow books, make reservations, and incur fines.

- **Attributes:**

- `String id`: The unique identifier for the patron.
- `String name`: The name of the patron.
- `List<Book> borrowedBooks`: A list of books currently borrowed by the patron.
- `List<Reservation> reservations`: A list of reservations made by the patron.
- `List<Fine> fines`: A list of fines incurred by the patron.

- **Key Methods:**

- `borrowBook(Book book)`: Adds a book to the list of books borrowed by the patron.
- `returnBook(Book book)`: Removes a book from the list of books borrowed by the patron.
- `addReservation(Reservation reservation)`: Adds a reservation to the patron's list.
- `removeReservation(Reservation reservation)`: Removes a reservation from the patron's list.
- `addFine(Fine fine)`: Adds a fine to the patron's list.
- `removeFine(Fine fine)`: Removes a fine from the patron's list.

2.4 Inventory

The `Inventory` class manages the quantity of each book in the library.

- **Attributes:**

- `Map<String, Integer> books`: A map of book ISBNs to their quantities.

- **Key Methods:**

- `addBook(Book book, int quantity)`: Adds a specified quantity of a book to the inventory.
- `removeBook(Book book, int quantity)`: Removes a specified quantity of a book from the inventory.
- `getQuantity(Book book)`: Returns the quantity of a specific book in the inventory.

2.5 Reservation

The ``Reservation`` class represents a reservation made by a patron for a specific book.

- **Attributes:**

- ``String id``: The unique identifier for the reservation.
- ``Book book``: The book being reserved.
- ``Patron patron``: The patron who made the reservation.
- ``Date reservationDate``: The date when the reservation was made.

- **Key Methods:**

- ``cancel()``: Cancels the reservation by removing it from the patron's list of reservations.

2.6 Fine

The ``Fine`` class represents a fine imposed on a patron for overdue books or other violations.

- **Attributes:**

- ``String id``: The unique identifier for the fine.
- ``Patron patron``: The patron who owes the fine.
- ``double amount``: The amount of the fine.
- ``String reason``: The reason for the fine.

- **Key Methods:**

- ``pay(double paymentAmount)``: Processes a payment towards the fine.

3. Data Management

3.1 In-Memory Storage

The system uses in-memory data structures to manage books, patrons, reservations, and fines. This approach is suitable for a simplified model and allows for fast access to data. However, it also means that all data is lost when the application is terminated.

- **Books:** Managed as a list within the ``Library`` class.
- **Patrons:** Managed as a map (ID -> Patron) within the ``Library`` class.
- **Inventory:** Managed as a map (ISBN -> Quantity) within the ``Inventory`` class.
- **Reservations:** Managed as a priority queue within the ``Library`` class.
- **Fines:** Managed as a list within the ``Patron`` class.

3.2 Search and Sorting

The system provides basic search and sorting capabilities:

- **Search by ISBN:** Directly retrieves a book based on its ISBN using a linear search.
- **Search by Title:** Searches for books whose titles match or contain the search string.
- **Binary Search:** The ``SearchAlgorithms`` class provides a binary search function that operates on a sorted list of books by title.

3.3 Events Management

Library events are managed by storing them in a list within the ``Library`` class. Each event is represented by a ``LibraryEvent`` object, which contains details about the event.

4. User Interface

4.1 Command-Line Interface

The ``UserInterface`` class provides a simple command-line interface (CLI) for interacting with the system. It uses the ``Scanner`` class to read user input from the console.

- **Display Menu:** Shows the main menu with available options.
- **Collect Input:** Reads the user's input and returns it to the ``LibraryManager`` for processing.
- **Display Messages:** Outputs messages and results to the user.

4.2 Interaction Flow

The interaction flow is managed by the ``LibraryManager`` class:

1. **Display Main Menu:** The ``UserInterface`` displays the main menu.
2. **Collect User Input:** The user selects an option from the menu.
3. **Process Input:** The ``LibraryManager`` processes the input by calling the appropriate method on the ``Library`` class.
4. **Display Results:** The results of the operation are displayed to the user.

5. Technologies Used

5.1 Java SE

The entire application is developed using Java Standard Edition (SE). Key features used include:

- Collections Framework: For managing lists, maps, and queues.
- Scanner: For reading user input in the command-line interface.
- Comparable Interface: For sorting books by title.
- Date: For managing dates related to reservations and events.

5.2 JUnit 5

JUnit 5

is used for unit testing the application. Unit tests are written for all major components to ensure correctness and robustness. These tests are located in separate test classes corresponding to each component.

6. Development Practices

6.1 Modular Design

Each class in the system is designed to encapsulate a specific piece of functionality, making the system modular and easier to maintain. For example, the **`Library`** class manages the overall operations, while the **`Inventory`** class is solely responsible for tracking book quantities.

6.2 Test-Driven Development (TDD)

The development process followed Test-Driven Development (TDD) principles:

- Write Tests First: Unit tests were written before the corresponding code was implemented.
- Refactor with Confidence: With tests in place, the code was refactored and improved with confidence that the changes did not introduce new bugs.

7. Deployment

7.1 Running the Application

The application can be run as a standalone Java program. The entry point is the `LibraryManager` class:

```
public static void main(String[] args) {  
  
    LibraryManager manager = new LibraryManager();  
  
    manager.run();  
  
}
```

7.2 System Requirements

- **Java Version:** The application requires Java SE 8 or later.
- **Memory:** The system is lightweight and should run on any machine with at least 512 MB of RAM.

8. Future Enhancements

The current implementation is designed as a basic library management system. Future enhancements could include:

- **Persistent Storage:** Implementing a database for storing books, patrons, and transactions permanently.
- **GUI Interface:** Developing a graphical user interface (GUI) to replace the CLI, making the system more user-friendly.
- **Advanced Search:** Implementing more sophisticated search features, such as filtering by author, publisher, or publication date.
- **Notifications:** Adding a notification system to remind patrons about due dates or reserved books.

Conclusion

This technical documentation provides a comprehensive overview of the Library Management System's architecture, components, and technologies. It serves as a guide for developers and technical stakeholders to understand the system's inner workings and provides a solid foundation for further development and enhancements.

ADVANCED LIBRARY MANAGEMENT SYSTEM (ALMS) GROUP B

STREAMLINING AND AUTOMATING LIBRARY OPERATIONS

PRESENTED BY

EMMANUEL KOROMA – 32770

OYAH MARIAMA KAMARA – 55745

ABUBAKARR TARAWALLY - 55666

DATE: AUGUST 30, 2024

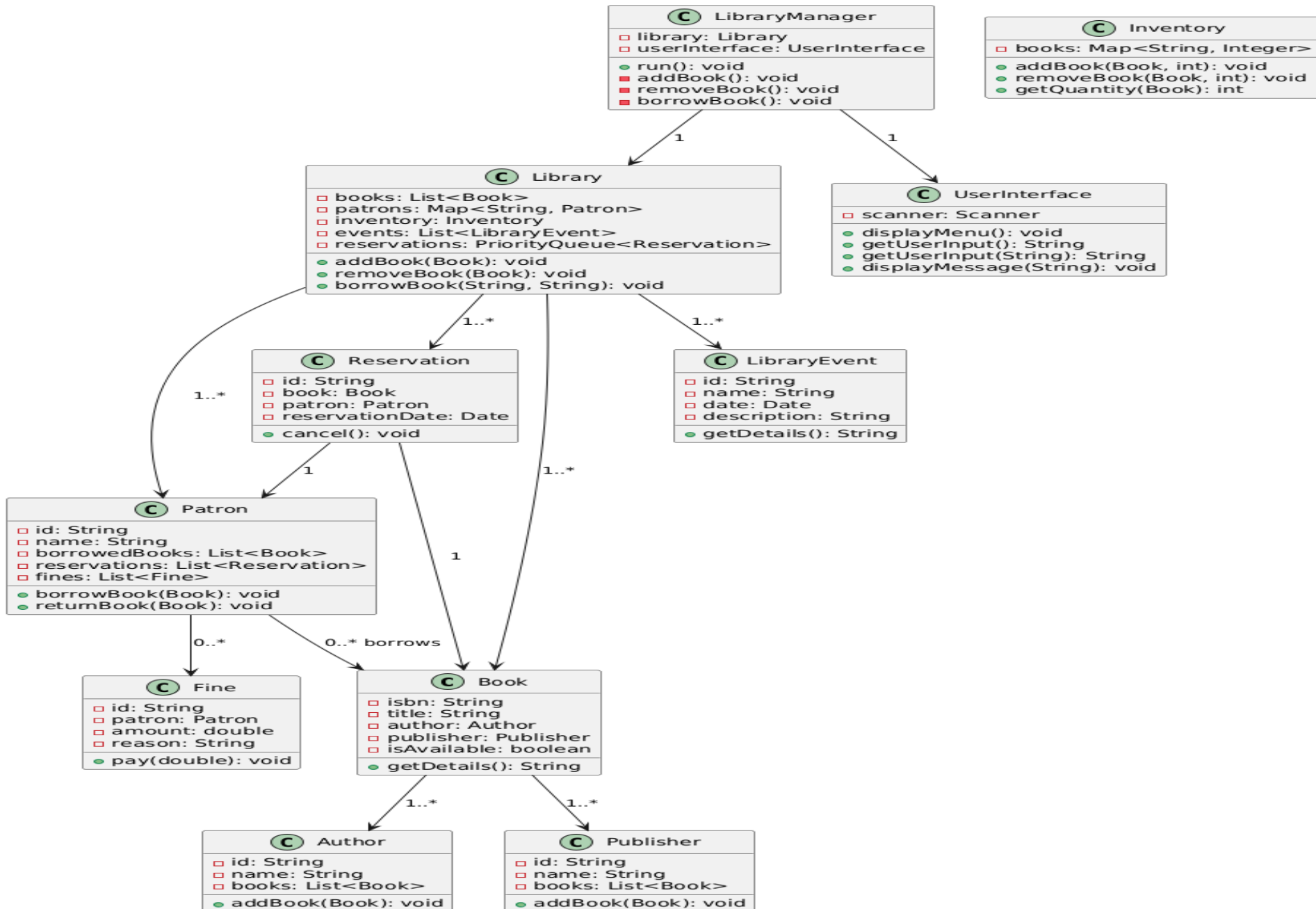
INTRODUCTION TO ALMS

- Overview of the Advanced Library Management System (ALMS).
 - Develop a robust system for managing library operations.
 - Implement efficient search and sorting algorithms.
 - Utilize advanced data structures.
 - Incorporate analytics features.
 - Ensure data integrity and security.

SYSTEM ARCHITECTURE

- Overview of the Class Diagram.
- Explanation of relationships between key components:
 - LibraryManager
 - Library
 - Patron
 - Inventory
 - Reservation
 - Fine

SYSTEM ARCHITECTURE



DATA FLOW IN ALMS

- High-level overview of how data flows through the system.
- Key processes involved:
 - Manage Books
 - Manage Patrons
 - Borrow/Return Books
 - Manage Reservations
 - Manage Fines
 - Generate Reports

DATA FLOW IN ALMS

- High-level overview of how data flows through the system.
- Key processes involved:
 - Manage Books
 - Manage Patrons
 - Borrow/Return Books
 - Manage Reservations
 - Manage Fines
 - Generate Reports

KEY ALGORITHMS IN ALMS

- Overview of key algorithms implemented in the system:
 - Linear Search for finding books.
 - Binary Search for efficient title searches.
 - Sorting algorithms for organizing books.
 - Priority Queue for managing reservations.

TECHNOLOGIES USED IN ALMS

- Overview of programming languages, data structures, and tools used:
 - Programming Language: Java
 - Data Structures: Arrays, ArrayList, LinkedList, HashMap, TreeSet
 - Algorithms: Binary Search, Merge Sort, Depth-First Search
 - Testing Framework: JUnit
 - Version Control: Git

CONCLUSION

- Summary of the benefits of ALMS:
 - Efficient library operations.
 - Enhanced user experience for both librarians and patrons.
 - Robust data management and security.
 - For further guidance, refer to the User Manual for troubleshooting and detailed instructions.